To-Do List REST API

A simple REST API built with **Node.js** and **Express** that performs CRUD operations (Create, Read, Update, Delete) on a list of tasks. Tasks are stored in memory for now, so they reset when the server restarts. A database (MongoDB) will be added in a later stage.

Features

- Create, view, update and delete tasks
- Input validation (title is required)
- Unique, auto-generated task IDs
- Proper HTTP status codes (200, 201, 400, 404)
- JSON requests and responses

Tech Stack

- Node.js
- Express.js

Project Structure

```
assig7/
└── backend/
    ├── server.js
    ├── package.json
    └── README.md
```

How to Run

1. **Install Node.js** (v18 or later) from https://nodejs.org
2. **Open a terminal** in the `backend` folder:

   ```
   cd backend
   ```

3. **Install dependencies:**

   ```
   npm install
   ```

   (If `express` is not listed in `package.json`, run `npm install express`.)
4. **Start the server:**

   ```
   node server.js
   ```

5. You should see:

```
Server is running on port 5000
```

6. Open `http://localhost:5000` in a browser. You should see `To-Do List API is running`.

To stop the server, press `Ctrl + C`.

API Endpoints

Base URL: `http://localhost:5000`

Method	Endpoint	Description	Success	Errors
GET	`/`	Health check	200	none
GET	`/tasks`	Get all tasks	200	none
GET	`/tasks/:id`	Get one task	200	404
POST	`/tasks`	Create a task	201	400
PUT	`/tasks/:id`	Update a task	200	400, 404
DELETE	`/tasks/:id`	Delete a task	200	404

Task object

```
{
  "id": 1,
  "title": "Learn Express",
  "description": "Create my first API",
  "completed": false
}
```

Create a task: `POST /tasks`

Request body:

```
{ "title": "Learn Express", "description": "Create my first API" }
```

Response (201):

```
{ "id": 1, "title": "Learn Express", "description": "Create my first API", "completed": false }
```

Error (400): `{ "message": "Task title is required" }`

Update a task: `PUT /tasks/:id`

Request body (all fields optional):

```
{ "title": "Learn JavaScript", "description": "Practice REST APIs", "completed": true }
```

Response (200): the updated task. Error (404): `{ "message": "Task not found" }`

Delete a task: `DELETE /tasks/:id`

Response (200): `{ "message": "Task deleted successfully" }`. Error (404): `{ "message": "Task not found" }`

How Task IDs Are Generated

A counter variable `nextId` starts at 1. Every time a task is created, the task gets the current value of `nextId` and the counter then increases by 1. IDs are never reused, even after a task is deleted. The counter resets when the server restarts because data is stored in memory.

Testing (PowerShell)

```
# Create a task
$new = @{ title = "Learn Express"; description = "First API" } | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:5000/tasks" -Method Post -ContentType "application/json" -
Body $new

# Get all tasks
Invoke-RestMethod -Uri "http://localhost:5000/tasks" -Method Get

# Get one task
Invoke-RestMethod -Uri "http://localhost:5000/tasks/1" -Method Get

# Update a task
$body = @{ title = "Learn JavaScript"; completed = $true } | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:5000/tasks/1" -Method Put -ContentType "application/json" -
Body $body

# Delete a task
Invoke-RestMethod -Uri "http://localhost:5000/tasks/1" -Method Delete
```

Note: restarting the server clears all tasks, so create a task first after every restart.

Future Improvements

- Store tasks in MongoDB using Mongoose
- Add a React frontend
- Add authentication and pagination